Creative Media Analyzer

Methodology

Variable extraction, group-by analysis, and the planned regression path

Author Theo Rajan

Date 2026-05-21

Deployment https://media-analyzer-theta.vercel.app

Repository github.com/theodoreheroldrajan-sketch/media-analyzer

Contents

1. Overview

The Creative Media Analyzer turns a folder of ad images plus a performance CSV into a structured comparison of which creative patterns correlate with each performance metric. The pipeline is:

- **Ingest.** Images uploaded to Supabase Storage, performance rows parsed from CSV with auto-detected column mapping.
- **Match.** Six-method cascade linking each image to a performance row (exact filename, filename minus extension, embedded platform ID, prefix, contains, fuzzy Levenshtein).
- **Extract.** Claude Haiku 4.5 vision call with forced tool_use produces a structured variable record per image, conforming to a project-versioned JSON schema.
- **Analyze.** Lite path runs descriptive group-by statistics with sample-size confidence. Pro path adds the regression layer (designed in this paper; mocked in the demo; production backend pending real-data validation).
- **Surface.** Dashboard with metric switcher, trust score, variable explorer, ranked creative gallery, and insights panel.

2. Variable extraction

Schema construction. The extraction API builds an Anthropic Tool object dynamically from the project's `variable_schemas` row. Each variable definition declares a name, type (boolean, enum, integer, string), an optional enum list, and a description. The Tool's input_schema is assembled at runtime and the request uses `tool_choice: { type: "tool", name: "extract_creative_variables" }` to force structured output.

Why tool_use. Free-text image captions cannot be grouped or compared. Tool_use commits the model to a schema the rest of the pipeline can rely on. The cost is rigidity: a value outside the enum is rejected. The benefit is that the database, the matcher, the dashboard, and the future regression all share a single contract.

Schema design. Two tiers ship today.

- **Universal variables (24).** Twelve boolean (product_visible, human_present, face_visible, text_overlay, logo_visible, brand_colours_used, headline_present, cta_present, offer_present, price_shown, urgency_cue, social_proof). Eight enum (creative_format, aspect_ratio, colour_palette, contrast, text_density, visual_clutter, message_angle, funnel_stage). One integer (number_of_people). Three string (primary_visual_subject, cta_text, primary_hook). Strings are extracted for completeness but excluded from analysis as high-cardinality fields.
- **Category templates (10).** Each loads a small set of vertical-specific variables. Ecommerce adds product_category, discount_percentage, free_shipping_mentioned, product_count, lifestyle_vs_studio, user_generated_content. SaaS adds screenshot, feature_highlighted,

demo_or_free_trial, social_proof_type, persona_targeted. Comparable sets exist for fintech, food_delivery, gaming, dating, education, health_fitness, travel, real_estate. A generic fallback covers categories without a dedicated template.

Brand context injection

Each extraction call is prefaced with a system string assembled from the project's setup row:

```
Brand: {brand}, Category: {category}, Platform: {platform}, KPI: {kpi}, Goal: {goal},
Audience: {audience}
```

This is the v1 implementation of the planned knowledge-base layer. It is enough for the model to disambiguate genre signals (skincare warmth versus fintech minimalism, for instance) but does not yet learn from previous campaigns.

3. Lite methodology (production)

All methods in this section are live in production for any dataset under 100 creatives. Source: `src/lib/analytics.ts`.

3.1 Method - volume-weighted aggregation

Question. What is the metric value for a group of creatives sharing a variable value?

Approach. Sum raw counters across the group, then compute the metric from the sums - not an average of per-creative rates.

```
CTR_group = (sum(clicks) / sum(impressions)) * 100
CPC_group = sum(spend) / sum(clicks)
CPA_group = sum(spend) / sum(conversions)
CVR_group = (sum(conversions) / sum(clicks)) * 100
ROAS_group = sum(revenue) / sum(spend)
```

Rationale. A creative with 100,000 impressions and 200 clicks should not be weighted equally with one that had 500 impressions and 50 clicks. Summing raw values weights by volume implicitly. The alternative (mean of per-creative CTRs) inflates the influence of low-volume outliers.

Assumption. Each creative's raw counters are correctly attributed - that the CSV row genuinely covers the period of interest and includes all relevant impressions/clicks/spend. Multi-row creatives (the same image running in three ad sets, producing three CSV rows) violate this; the current solution is documented in the instructions page rather than enforced in code.

3.2 Method - percentage delta vs overall

Question. Is this group's metric better or worse than the dataset average?

Approach.

```
delta = ((group_metric - overall_metric) / overall_metric) * 100
```

Rationale. Marketers think in percentage uplift, not absolute differences. A +20% CTR delta is immediately interpretable; an absolute delta of 0.4 percentage points is not.

Where this breaks. No standard error. A +20% delta from a group of three creatives is treated identically to a +20% delta from thirty. The confidence label (next section) is the partial mitigation; a proper standard error on the delta is regression territory.

3.3 Method - sample-size confidence labels

Question. How seriously should the reader take this row?

Approach. A four-level discrete label based on group size n:

n	Label	Treatment
less than 3	insufficient	Excluded from charts; shown with placeholder values in the variable table for transpa
3-4	low	Treat as hypothesis only
5-9	medium	Directional
10 or more	high	Reliable for decision-making

Rationale. A discrete label is more honestly read by non-statisticians than a continuous p-value. The thresholds are conservative rules of thumb rather than calibrated against a specific power calculation.

Known weakness. The label ignores impression volume. Three creatives each delivering 100,000 impressions are labelled "low" while ten creatives each delivering 100 impressions are labelled "high". The former arguably has more statistical power. A future iteration may weight by impressions or move to a continuous confidence score.

3.4 Method - composite trust score

Question. How much should the reader trust the dashboard before reading any single number?

Approach. A floor-gated composite of five 0-100 sub-scores, rounded to a single 0-100 dataset-level value. Three of the sub-scores are floor conditions; the other two contribute proportionally on top.

Sub-score	Role	Formula
Creative count	floor	min(100, n / 50 * 100)
Mapping quality	floor	confirmed_mappings / total * 100
Data completeness	floor	creatives_with_impressions_and_spend / total * 100
Volume (impressions)	upper (0.5)	min(100, log10(impressions) / 6 * 100)
Bucket balance	upper (0.5)	(total_groups - n_under_3) / total_groups * 100

```
floor_score = min(creative_count, mapping_quality, data_completeness)
upper_score = volume_score * 0.5 + bucket_balance * 0.5
trust_score = round(floor_score * (upper_score / 100))
```

Levels: 80-100 Excellent, 60-79 Good, 40-59 Fair, under 40 Poor.

Rationale. This is a heuristic, not a statistical measure. Its job is to set the prior of trust before the user reads the per-row numbers. The floor-gating closes a failure mode the earlier weighted-average composite had: a low mapping rate could be averaged away into a misleadingly good overall score. With the floor design, the worst floor sub-score caps the headline number, and the user always sees the constraint.

Removed sub-score. An earlier revision had a sixth sub-score derived from `extraction_results.confidence`. The Anthropic tool_use API does not return a self-confidence value, so that column was never populated; the sub-score was effectively constant. The column and the sub-score were both removed on 2026-05-15.

Limit of the design. The score does not detect adversarial or systematically biased data. A dataset where one campaign accounts for 90% of spend can still score Excellent on every sub-score, despite being practically a single-creative analysis.

4. Pro methodology (designed; demo-only at present)

Honest status. The Pro dashboard ships in the interactive demo with mocked statistics. The mocked numbers are generated by `src/lib/demo-data.ts` from the actual group-by deltas plus seeded PRNG noise, so they look realistic and internally consistent across the demo, but they are not the output of a real regression. The real backend is designed below and will be built against the first 100-plus creative real dataset.

Why mock rather than implement upfront. Decisions about regularization, interaction terms, and weighting are difficult to make in the abstract. They depend on the empirical structure of the first real dataset (multicollinearity in the predictors, fitting behavior, residual patterns). Synthesizing those choices on fake data risks shipping a model that does not survive contact with real distributions.

4.1 Designed model - multiple linear regression

Form.

```
Y_i = b0 + b1 * X_1i + b2 * X_2i + ... + bk * X_ki + e_i
```

Y is the per-creative metric (CTR, CPC, CPA, CVR, or ROAS). Predictors are the extracted variables, encoded as follows: booleans as binary dummies, enums as one-hot with one reference category dropped, integers as continuous, strings excluded.

4.2 Interaction terms

Plain main-effects regressions miss the patterns marketers most want to surface ("does human-in-frame interact with urgency cue?"). Three interaction families are flagged in the design:

- Boolean times Boolean (for example human_present times urgency_cue).
- Boolean times Enum (for example cta_present times message_angle).
- Boolean times Continuous (for example human_present times number_of_people).

Selection strategy: start main-effects-only, then add interactions among the top 5-10 main effects by t-statistic. Compare adjusted R-squared and AIC/BIC. LASSO is the fallback if predictor count blows up past 50 after dummy encoding.

4.3 Weighted least squares

Impression-weighted regression is the planned default rather than vanilla OLS. A creative with 100,000 impressions yields a CTR estimate that is roughly 10x more precise than one with 1,000 impressions. The weight is impression count or sqrt(impressions) - the choice is sensitive to outliers and will be tuned empirically on real data.

4.4 Outputs surfaced

Mocked in the demo today and planned for the production implementation:

- Per-coefficient: beta, standard error, t-statistic, p-value, 95% confidence interval, standardized beta, significance flag at $p < 0.05$.
- Model-level: R-squared, adjusted R-squared, model F-test p-value, maximum VIF (multicollinearity check).
- Interaction matrices: small heatmap grids showing the metric across paired variable values, with cell counts.
- Insights panel: technical phrasing referencing beta and p in copy ("face_visible shows the largest standardized coefficient on CTR, beta = +0.34, $p < 0.001$").

4.5 Client-side simple OLS (live and real)

One small piece of real regression code is live today: the scatter chart fits a single-predictor OLS line client-side using the standard normal equations. Source: `src/components/dashboard/charts/regression-chart.tsx`. It computes slope, intercept, and R-squared on the visible scatter (x = log10 impressions, y = chosen metric) and draws a fitted line plus an R-squared label. This is a visualization aid, not an inferential output. The Pro dashboard's main regression table is mocked; this single-predictor line is real.

5. The Lite/Pro boundary

Rule. 100 creatives. Hard threshold set in `src/app/api/dashboard/route.ts`, line 237 (`regressionReady: creativeData.length >= 100`).

Justification. The rule of thumb for stable OLS estimates is 5-10 observations per predictor. With approximately 25 active candidate predictors (about 18 enabled variables after one-hot encoding adds dummies), the floor is 100. This is conservative; at the lower end (about 100 observations, about 25 predictors) regularization or interaction restraint becomes important, and standard errors widen.

What changes at the boundary.

- Lite dashboard adds the regression-table panel and the interaction-matrix panel.
- The variable explorer gains four new chart types (scatter, regression, distribution, heatmap) beyond the bar chart.
- The mapping page upgrades from flat table to match cards with confidence and method badges, plus separate sections for suggested and unmatched.
- The variables page upgrades from a toggle list to a four-tier schema builder including AI suggestions and a custom variable form.
- The insights panel switches from plain-English cards to technical cards that reference coefficients and p-values inline.

6. Limitations

Mandatory section. The honest version of this tool's claims, in one place. See `LIMITATIONS.md` in the repository for a longer, code-grounded version including operational gaps.

- **No formal hypothesis testing in Lite production.** Lite output is descriptive. Bootstrap 95% confidence intervals are computed on every delta, but they're displayed as visual whiskers rather than fed into a significance test.
- **Limited multiple-comparison correction.** The Pro regression table applies Benjamini-Hochberg FDR to exploratory variables, and the Lite variable performance table now sorts exploratory rows by noise-adjusted effect size (|delta| times sqrt(n)) by default. The Lite group-by deltas themselves remain uncorrected point estimates with CIs; family-wise comparison rate is unaddressed in Lite by design.
- **No interaction effects in Lite production.** Each variable is analyzed independently in Lite. Pairs that matter together are invisible until Pro mode ships with real regression.
- **Confidence label ignores impression volume.** n=3 creatives delivering 300,000 impressions each are labelled "low". n=10 creatives delivering 1,000 impressions each are labelled "high". The former has more statistical power. On the roadmap to fix.

- **Missing values are silently excluded.** A creative with a null variable is dropped from that variable's group. If missingness correlates with something (for example, cta_text is null when there is no CTA), the resulting groups are biased.
- **No temporal handling.** All data is treated as a single cross-section. Seasonal effects, fatigue, and platform algorithm changes are unmodeled. Planned for v3 once real multi-snapshot data is available.
- **Correlation, not causation.** Every dashboard output is correlational. The tool is built to generate informed hypotheses for the next brief, not to prove that any variable causes performance changes.
- **Scale ceiling.** Streaming AI extraction over Vercel Hobby is practical up to roughly 150-200 creatives per run. Above that, batching and resumability become necessary.
- **Pro statistics in the demo are mocked.** The numbers in the regression table in the demo are deterministic noise added to the group-by deltas. They are designed to look right, not to be right. The methodology paper is the canonical reference for what those numbers will be once the production backend ships.
- **Bootstrap CI is not impression-weighted.** Resampling is uniform across creatives. A statistically rigorous volume-weighted bootstrap would resample with probability proportional to impressions. Acceptable approximation in current scope; the Pro production OLS will use WLS instead.

7. References

In-repo. `ANALYSIS_METHODOLOGY.md` in the project root contains the canonical version of this paper - longer technical specification of the methods above, including detailed formulas, weighted least squares notes, and a complete trust-score breakdown. The PDF is the portfolio-facing condensation of that document.

External. Standard OLS treatment (any econometrics textbook). Rule-of-thumb of 5-10 observations per predictor is widely cited in regression introductions; the specific threshold for this project was chosen as a conservative floor rather than derived from a power calculation.

8. Open questions for review

These are the genuine methodological uncertainties. Answers will shape the production OLS backend when real data arrives. If you have informed opinions on any of these, open a GitHub discussion on the repo or file an issue.

- **Sample size threshold.** Is 100 creatives sufficient for OLS with ~25 predictors plus interactions? Should the threshold increase, or should regularisation kick in earlier?
- **Weighting.** Should the tool weight by impressions (WLS) in the group-by analysis as well, not just regression? Currently all creatives are treated equally in Lite.

- **Multiple comparisons.** For the current group-by approach, should the tool apply a Bonferroni correction or FDR control to the delta rankings, even without formal p-values?
- **Variable selection.** With 24 universal plus 4-6 category variables, plus one-hot encoding, the model could have 60-plus predictors. What regularisation approach makes most sense?
- **Temporal effects.** If the operator uploads a new CSV each month, should the tool model time as a fixed effect or treat each upload as a separate cross-section?
- **Clustering.** Creatives within the same campaign or ad set may share characteristics. Clustered standard errors or a mixed-effects model?
- **Non-linear effects.** For integer variables like `number_of_people`, should the tool include polynomial terms or treat them as categorical?
- **Practical significance.** What minimum effect size (in the metric's units, e.g. +0.2% CTR) should the tool flag as "actionable" vs "statistically significant but negligible"?